Design pattern Decorator

AWEB

Daniel, Joao, Nathan

Le design pattern Decorator **3**
Description 3
Exemple 3
Structure 3
Domaines d’utilisation 4
Implémentation PHP 4

Le design pattern Decorator

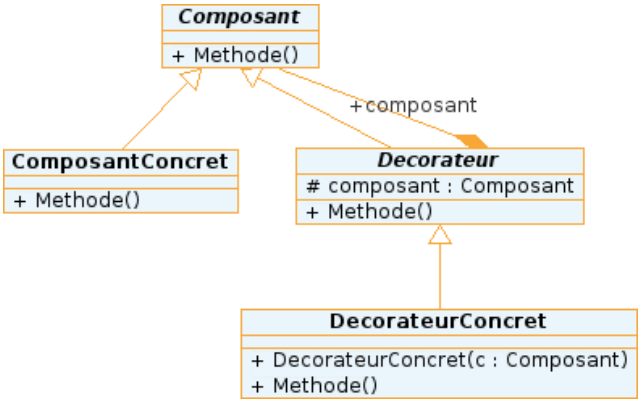
Description

Le design pattern Décorateur (ou Wrapper) est un modèle structurel qui permet d'ajouter dynamiquement de nouvelles fonctionnalités à un objet existant sans modifier sa structure ni son code source. Il utilise la composition plutôt que l'héritage, emballant l'objet original dans un objet “décorateur” qui implémente la même interface.

Exemple

« Imaginez une simple tasse de café. Si vous souhaitez y ajouter du lait, du sucre ou de la crème fouettée, vous ne créez pas une nouvelle sous-classe « CaféAuLait » ou « CaféAuSucreFouetté » pour chaque combinaison. Au lieu de cela, vous partez du café nature et l'agrémenter en ajoutant les ingrédients un à un. Le café au lait reste fondamentalement un café (il peut être servi partout où l'on a besoin de café), mais il possède désormais une caractéristique supplémentaire : le goût du lait. Vous pouvez ensuite l'agrémenter de sucre, et ainsi de suite. Chaque ajout confère au café un nouveau comportement (modifiant son goût ou son prix) sans altérer la classe Café elle-même. En informatique, le modèle Décorateur fonctionne de la même manière : on prend un objet et on l'enveloppe dans un ou plusieurs objets décorateurs qui ajoutent des fonctionnalités tout en préservant l'interface d'origine. » - Maxim Gorim (Medium.com)

Structure



Domaines d’utilisation

Quand on veut implémenter un système de log sans affecter le fonctionnement de code existant.

Implémentation PHP

```php
interface IComponent {
    public array $classList { get; set; }
    public string $textContent { get; set; }

    public function toHTML() : string;
}
```

Interface de base des composant HTML de l'application servant à mettre en place le design pattern décorateur

```php
class ComponentStyle implements IComponent {
    protected IComponent $component;
    public array $classList {
        get => $this->component->classList;
        set (array $value) => $this->component->classList = $value;
    }
    public string $textContent {
        get => $this->component->textContent;
        set(string $value) => $this->component->textContent = $value;
    }

    public function __construct(IComponent $component, array $classList) {
        $this->component = $component;
        $this->classList = array_merge($this->classList, $classList);
    }

    public function toHTML(): string {
        return $this->component->toHTML();
    }

    public function __toString() {
        return $this->toHTML();
    }
}
```

ComponentStyle est la class parent de tous les décorateurs, elle sert à ajouter des classes tailwindcss au composant de base afin de changer son style

```php
class Alert extends Component {
    public AlertType $type;
    public string $message;

    public function __construct(string $message = 'Alert', AlertType $type = AlertType::INFO)
    {
        parent::__construct();
        $this->type = $type;
        $this->message = $message;
        $component = new BaseBorder($this);
        $component = new Color($component, $type->value. '-200', "black");
        $component = new ComponentStyle($component, ["h-fit", "w-full", "p-2", "flex", "justify-between", "mb-2"]);
        $this->classList = array_merge($this->classList, $component->classList);
    }

    public function toHTML(): string
    {
        $js = 'this.parentElement.remove()';
        $classes = implode(' ', $this->classList);
        return "
            <div class='$classes'>
                <p>$this->message</p>
                <button onclick='$js'>
                    <i class='bi bi-x-lg'></i>
                </button>
            </div>
        ";
    }
}
```

Un exemple de composant Alert, on utilise le design pattern afin de lui donner la bordure défini par le thème avec le décorateur BaseBorder, on lui donne une couleur avec le Color décorateur et on ajoute du style plus précis grâce au décorateur ComponentStyle de base